# El pensamiento computacional

Nombre: _____ Fecha: _____

## Qué vas a aprender

- Explicar qué es el pensamiento computacional y para qué sirve
- Aplicar la descomposición, el reconocimiento de patrones, la abstracción y los algoritmos a problemas cotidianos
- Resolver retos desenchufados sin ordenador
- Valorar el error como parte del proceso de resolver problemas

## Para pensar

**Organizar una fiesta de cumpleaños para toda la clase parece un lío enorme. ¿Cómo lo harías para que no se te olvide nada?**

## Explicación

### ¿Qué es el pensamiento computacional?

El pensamiento computacional es una forma de pensar para **resolver problemas** de manera ordenada, inspirada en cómo trabajan los informáticos. Consiste en analizar un problema, dividirlo, encontrar lo que tiene en común con otros, quedarse con lo esencial y diseñar una solución paso a paso que podría seguir una persona o una máquina. Aunque se llame «computacional», **no hace falta un ordenador** para usarlo: lo aplicamos al preparar una receta, organizar la mochila, planificar un viaje o resolver un problema de matemáticas. Lo que sí ocurre es que es la forma de pensar necesaria para **programar**: antes de escribir una sola línea de código, hay que haber entendido el problema y pensado la solución. Se apoya en cuatro pilares: la **descomposición**, el **reconocimiento de patrones**, la **abstracción** y los **algoritmos**. Además, incluye una actitud: probar, equivocarse, aprender del error y volver a intentarlo.




### Descomposición

La descomposición consiste en **dividir un problema grande y complejo en partes más pequeñas** y fáciles de resolver. Cada parte se resuelve por separado y, al juntarlas, se tiene la solución completa. Es lo que hacemos al limpiar una habitación muy desordenada: en vez de agobiarnos, decidimos recoger primero la ropa, luego los libros, después la mesa y al final barrer. En programación, un videojuego se descompone en partes: los personajes, el escenario, el movimiento del protagonista, los enemigos, los puntos, los niveles, la pantalla final... Y cada una de esas partes, si sigue siendo difícil, se puede volver a dividir. La descomposición tiene otra ventaja: permite **repartir el trabajo en equipo**, porque cada persona puede encargarse de una parte. Al descomponer hay que

tener cuidado de no olvidar ninguna parte y de pensar cómo encajarán luego entre sí.




## Reconocimiento de patrones

Un patrón es algo que **se repite** o que se parece entre distintos problemas o dentro de un mismo problema. Reconocer patrones permite **reutilizar soluciones** que ya conocemos y ahorrar trabajo. Por ejemplo, si sabemos cómo se dibuja un cuadrado (avanzar y girar, cuatro veces), enseguida vemos que un triángulo o un hexágono siguen el mismo patrón, cambiando solo el número de lados y el ángulo de giro. En un juego, todos los enemigos se mueven de forma parecida, así que basta con programar uno y copiarlo. En las matemáticas, las series de números siguen patrones (2, 4, 6, 8...). Y en la vida diaria, si sabemos llegar en bici al instituto, ese conocimiento nos sirve para llegar a la biblioteca, que está cerca. En programación, los patrones que se repiten se convierten en **bucles** («repetir») o en **bloques propios** que se pueden usar muchas veces.

undefined

## Abstracción

La abstracción consiste en **quedarse con lo importante** de un problema e ignorar los detalles que no influyen en la solución. Un ejemplo clásico es el plano del metro: no muestra las calles, los edificios ni las distancias reales, sino solo las estaciones, el orden en que están y las líneas que las conectan, que es lo único que necesita un viajero. Si mostrara todo, sería imposible de leer. Del mismo modo, para programar un juego de fútbol no hace falta saber el color de las botas de cada jugador, pero sí su posición, su velocidad y quién tiene el balón. La abstracción permite crear **modelos simplificados** de la realidad que una

máquina puede manejar. Hacer una buena abstracción no es fácil: hay que pensar bien qué datos son **imprescindibles** y cuáles se pueden descartar. Si quitamos demasiado, la solución no funcionará; si dejamos demasiado, será complicada.

| Problema | Lo importante | Lo que se puede ignorar |
|---|---|---|
| Ir en metro a un sitio | estaciones, líneas, transbordos | calles, distancias exactas |
| Juego de fútbol | posición, velocidad, balón | color de las botas, el público |
| Receta de cocina | ingredientes, cantidades, pasos | la marca de la sartén |
| Mapa del tiempo | temperatura, lluvia, viento | forma exacta de las nubes |

undefined

## Algoritmos

Un algoritmo es una **secuencia ordenada y finita de pasos** para resolver un problema o hacer una tarea. Una receta de cocina, las instrucciones de montaje de un mueble o las indicaciones para llegar a un lugar son algoritmos. Para que un algoritmo funcione, sus pasos deben ser **claros y precisos** (sin ambigüedad: «añade un poco de sal» no es preciso; «añade 5 gramos de sal» sí), estar en el **orden correcto** y **terminar** en algún momento. Un mismo problema puede resolverse con distintos algoritmos, unos más eficientes que otros. Un **programa** no es más que un algoritmo escrito en un lenguaje que entiende el ordenador. Por eso, antes de programar, conviene escribir el algoritmo con palabras o con un dibujo, y comprobar que funciona «ejecutándolo» a mano, como si fuéramos la máquina. Los algoritmos se estudian con más detalle en el siguiente tema.

undefined

## Actividades desenchufadas

Una forma divertida de practicar el pensamiento computacional es hacer actividades desenchufadas, sin ordenador. Por ejemplo, el **robot humano**: en una

cuadrícula dibujada en el suelo o en papel, una persona hace de «robot» y solo puede obedecer órdenes muy simples: «avanza una casilla», «gira a la derecha», «gira a la izquierda», «coge». Otra persona escribe el programa con flechas para llevar al robot desde la salida hasta un tesoro, evitando obstáculos. Si el robot acaba en otro sitio, hay que **buscar el error** en la secuencia de flechas. Otras actividades son dibujar una figura siguiendo las instrucciones de un compañero sin verla (y comprobar lo difícil que es dar instrucciones precisas), ordenar las tarjetas de un proceso mezclado, codificar un dibujo en una cuadrícula con números o adivinar un número con preguntas de «mayor o menor». Estas actividades enseñan a pensar como un programador sin preocuparse por la tecnología.



## Evaluar y aprender del error

Una parte esencial del pensamiento computacional es la evaluación: comprobar si la solución funciona de verdad, si es correcta en todos los casos y si se podría mejorar. Para ello se prueba con distintos ejemplos, incluidos los casos extraños («¿y si el número es cero?», «¿y si el personaje llega al borde?»). Cuando algo falla, toca depurar: localizar el error, entender por qué ocurre y corregirlo. Equivocarse no es un fracaso, sino una pista para aprender. Los programadores profesionales dedican mucho tiempo a probar y depurar. También conviene **comparar soluciones**: dos algoritmos pueden llegar al mismo resultado, pero uno puede necesitar menos pasos o ser más fácil de entender. Finalmente, el pensamiento computacional también implica **generalizar**: una vez resuelto un problema, preguntarse si la misma solución sirve para otros parecidos. Perseverancia, paciencia, trabajo en equipo y creatividad son tan importantes como la lógica.

undefined

## El pensamiento computacional en ocho ideas



## Ejercicios

1. **¿Qué es el pensamiento computacional?**

2. **Escribe los cuatro pilares del pensamiento computacional.**

3. **¿Qué es descomponer? Pon un ejemplo.**

4. **¿Qué es un patrón? Pon un ejemplo.**

5. **¿Qué es la abstracción? Pon un ejemplo.**

6. **¿Qué características debe tener un algoritmo?**

7. **Escribe un algoritmo para lavarse los dientes.**

8. **¿Qué es una actividad desenchufada? Pon un ejemplo.**

9. **¿Qué significa depurar?**

10. **¿Por qué es útil reconocer patrones?**

## Practica

1. **Dividir un problema en partes es...**
   - **A)** descomposición
   - **B)** abstracción
   - **C)** depuración
2. **Un plano del metro es un ejemplo de...**
   - **A)** abstracción
   - **B)** bucle
   - **C)** patrón
3. **Una receta es un...**
   - **A)** algoritmo
   - **B)** patrón
   - **C)** sensor
4. **Lo que se repite en varios problemas es un...**
   - **A)** patrón
   - **B)** error
   - **C)** dato
5. **Para usar el pensamiento computacional...**
   - **A)** no hace falta ordenador
   - **B)** hace falta internet
   - **C)** hace falta un robot
6. **«Añade un poco de sal» es una instrucción...**
   - **A)** ambigua
   - **B)** precisa
   - **C)** repetitiva
7. **Corregir errores se llama...**
   - **A)** depurar
   - **B)** descomponer
   - **C)** abstraer
8. **Un pentágono regular gira en cada vértice...**
   - **A)** 72°

**B)** 90°

**C)** 60°

9. **Un algoritmo debe...**

**A)** terminar

**B)** ser infinito

**C)** ser ambiguo

10. **Usar una solución conocida en un problema parecido es...**

**A)** generalizar

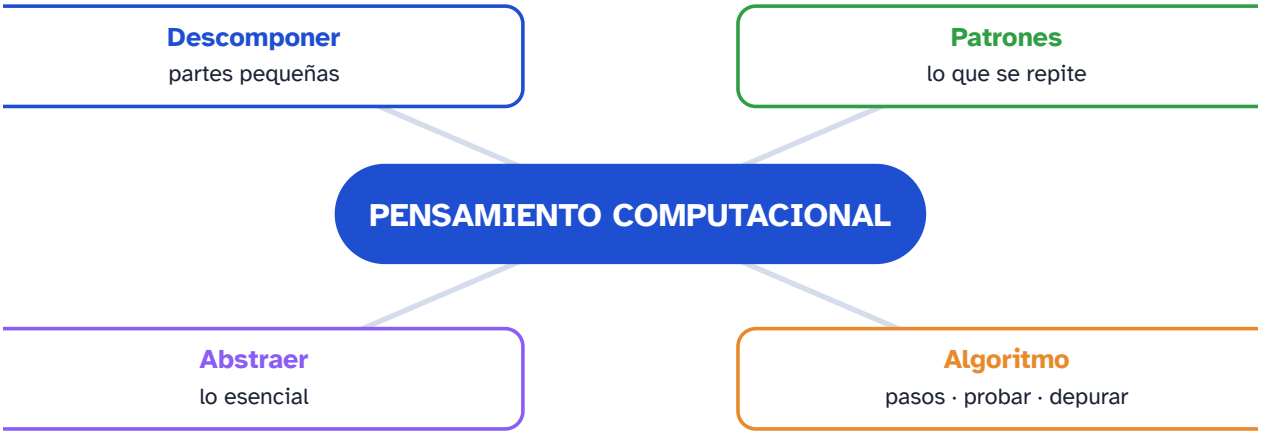
**B)** olvidar

**C)** descomponer

## Explícalo con tus palabras

Incluye: Cuáles son los cuatro pilares del pensamiento computacional, con un ejemplo de cada uno; Por qué la abstracción ayuda a resolver problemas; Por qué equivocarse forma parte de programar.

## Resumen



## Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

**Pensamiento computacional**

Forma ordenada de resolver problemas, como un informático.

**Descomposición**

Dividir un problema en partes más pequeñas.

**Patrón**

Algo que se repite o se parece.

**Abstracción**

Quedarse con lo esencial e ignorar el resto.

**Algoritmo**

Secuencia ordenada y finita de pasos.

**Depurar**

Encontrar y corregir errores.

**Actividad desenchufada**

Ejercicio de programación sin ordenador.

**Generalizar**

Aplicar una solución a problemas parecidos.

## Soluciones

1. **A)** Descomposición.
2. **A)** Abstracción.
3. **A)** Algoritmo.
4. **A)** Patrón.
5. **A)** Se puede hacer sin ordenador.
6. **A)** No es precisa.
7. **A)** Depurar.
8. **A)** 360 ÷ 5.
9. **A)** Ser finito.
10. **A)** Generalizar.

## Soluciones de los ejercicios

1. Una forma de pensar para resolver problemas de manera ordenada, como lo haría un informático.
2. Descomposición, reconocimiento de patrones, abstracción y algoritmos.
3. Dividir un problema en partes más pequeñas; organizar una fiesta en invitaciones, comida, música y juegos.
4. Algo que se repite o se parece; dibujar polígonos repitiendo avanzar y girar.
5. Quedarse con lo importante e ignorar lo demás; un plano del metro.
6. Pasos claros y precisos, en orden y que terminen.
7. Poner pasta en el cepillo, mojarlo, cepillar arriba y abajo dos minutos, escupir, enjuagar la boca y el cepillo, guardarlo.
8. Una actividad de pensamiento computacional sin ordenador; el robot humano en una cuadrícula.
9. Encontrar y corregir errores en una solución.
10. Porque permite reutilizar soluciones conocidas y ahorrar trabajo.